

Problem chińskiego listonosza

Natalia Jezewska 263305

1 Wprowadzenie

Listonosz musi przejść każdą krawędź grafu co najmniej raz i wrócić do punktu startowego.

Pytanie: jaka jest minimalna łączna waga takiej trasy?

Trasa eulera, przechodząca każdą krawędź dokładnie raz, istnieje wtedy i tylko wtedy, gdy każdy wierzchołek ma parzysty stopień. Jeśli tak nie jest, musimy "podwoić" pewne krawędzie, żeby wyrównać stopnie.

2 Algorytm

1. Znalezienie wszystkich wierzchołków o nieparzystym stopniu.
2. Wyznaczenie najkrótszych ścieżek między każdą parą takich wierzchołków za pomocą algorytmu Dijkstra.
3. Znalezienie minimalnych skojarzeń doskonałych wśród wierzchołków o nieparzystym stopniu.
4. Dodanie do grafu krawędzi odpowiadających tym najkrótszym ścieżką, zduplikowanie ich.
5. Na tak zmodyfikowanym grafie znalezienie obwodu Eulera.
6. Koszt trasy to suma wag oryginalnych krawędzi i wag dodanych ścieżek.

3 Opis kodu

Kod składa się z dwóch klas `WeightedGraph` jako klasy bazowej i `ChinesePostman`, która po niej dziedziczy i rozszerza ją o algorytm główny. Pierwsza klasa reprezentuje nieskierowany graf ważony, słownik `graph` przechowuje listę sąsiedztwa, a `weight` wagi krawędzi, zapisane w obu kierunkach.

3.1 Metody

3.1.1 `WeightedGraph`

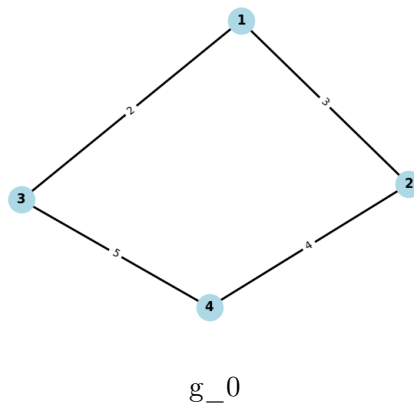
- `add_vertex(v)` - dodaje wierzchołek grafu, jeżeli jeszcze nie istnieje,
- `add_edge(edge, weight)` - dodaje krawędź nieskierowaną między dwoma wierzchołkami z podaną wagą,
- `__str__()` - zwraca tekstową reprezentację grafu, dla każdego wierzchołka podaje listę jego sąsiadów z wagami krawędzi,
- `plot()` - rysuje graf przy użyciu bibliotek `networkx` i `matplotlib`,
- `Dijkstra(s)` - wyznacza najkrótsze ścieżki od wierzchołka `s` do wszystkich pozostałych, używając kolejki priorytetowej. Zwraca słownik odległości `dist` oraz poprzedników `prev`, które pozwalają odtworzyć pełną ścieżkę

3.1.2 ChinesePostman

- `odd_degree_vertices()` - zwraca listę wierzchołków o nieparzystym stopniu,
- `_all_pairs_shortest_paths(odd_vertices)` - dla każdej pary wierzchołków nieparzystych wyznacza najkrótszą ścieżkę za pomocą Dijkstry,
- `_min_weight_matching(odd_vertices, sp)` - algorytmem Edmonsa dobiera optymalne pary wierzchołków nieparzystych tak, żeby łączny koszt zduplikowanych ścieżek był minimalny,
- `chinese_postman` - łączy wszystkie kroki algorytmu,
- `eulerian_circuit(extra_edges)` - wyznacza obwód Eulera algorytmem Hierholzera na grafie uzupełnionym o zduplikowane krawędzie i zwraca listę wierzchołków trasy.

3.2 Przykłady

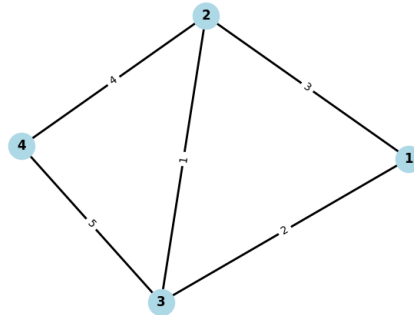
```
# graf eulrowski
g_0 = ChinesePostman(edges=[
    (1, 2, 3),
    (1, 3, 2),
    (2, 4, 4),
    (3, 4, 5)
])
g_0.plot()
```



```
cost, extra = g_0.chinese_postman()
circuit = g_0.eulerian_circuit(extra_edges=extra)
print("Trasa listonosza:", circuit)
```

```
Wierzchołki o nieparzystym stopniu: []
Graf eulrowski - przez każdą krawędź przejdzie dokładnie raz.
Koszt bazowy (suma wag krawędzi): 14
Dodatkowy koszt (zduplikowane ścieżki): 0
Minimalny koszt trasy listonosza: 14
Trasa listonosza: [1, 2, 4, 3, 1]
```

```
# graf poleulerowski
g_1 = copy.deepcopy(g_0)
g_1.add_edge([2,3], 1)
g_1.plot()
```

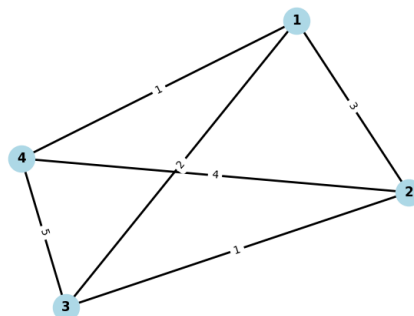


g_1

```
cost, extra = g_1.chinese_postman()
circuit = g_1.eulerian_circuit(extra_edges=extra)
print("Trasa listonosza:", circuit)
```

Wierzchołki o nieparzystym stopniu: [2, 3]
 Koszt bazowy (suma wag krawędzi): 15
 Dodatkowy koszt (zduplikowane ścieżki): 1
 Minimalny koszt trasy listonosza: 16
 Trasa listonosza: [1, 2, 4, 3, 2, 3, 1]

```
# graf nieeulerowski
g_2 = copy.deepcopy(g_1)
g_2.add_edge([1,4], 1)
g_2.plot()
```



g_2

```
cost, extra = g_2.chinese_postman()
circuit = g_2.eulerian_circuit(extra_edges=extra)
print("Trasa listonosza:", circuit)
```

Wierzchołki o nieparzystym stopniu: [1, 2, 3, 4]
Koszt bazowy (suma wag krawędzi): 16
Dodatkowy koszt (zduplikowane ścieżki): 2
Minimalny koszt trasy listonosza: 18
Trasa listonosza: [1, 2, 4, 1, 3, 2, 3, 4, 1]

Literatura

- [1] https://pl.wikipedia.org/wiki/Problem_chińskiego_listonosza - 05.06.2026r.
- [2] <https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/>
- 05.06.2026r.